

FinanceAI

Plan final para cerrar el MVP

Del estado real en GitHub al producto integrado - Fecha límite: sábado 22 de agosto por la noche

Para qué sirve este documento

Unifica las guías anteriores en un solo plan de trabajo. Está escrito para compartirlo con todo el equipo y responder tres preguntas: qué existe hoy, qué debe hacer cada frente y qué debemos poder demostrar el sábado por la noche.

Punto de partida revisado: ramas main, feature/integrar-frontend y frontend_fernando del repositorio G9-LATAM-Team-77, más las pruebas locales realizadas el 17 de agosto de 2026.

Regla principal de esta semana

No crear más piezas aisladas. Desde ahora todos trabajan desde la misma rama de integración, entregan cambios pequeños y prueban el flujo completo todos los días.

1. A dónde queremos llegar el sábado

El producto final no debe ser solamente un formulario que calcule un resultado. Debe sentirse como una aplicación personal donde cada usuario entra a su cuenta, guarda información y puede ver si su situación mejora o empeora con el tiempo.

- 1. El usuario crea una cuenta.
- 2. Inicia sesión con su correo y contraseña.
- 3. Ve únicamente sus propios datos.
- 4. Registra sus movimientos y datos financieros del período.
- 5. La información queda guardada en MySQL y no desaparece al cerrar la aplicación.
- 6. FinanceAI analiza el período actual y muestra perfil, gastos, categorías y recomendaciones.
- 7. El usuario puede volver después y consultar su historial.
- 8. El dashboard compara el período actual con el período anterior: gasto, ahorro, endeudamiento, perfil y principales categorías.
- 9. La solución funciona fuera de la computadora del desarrollador, desplegada en OCI.
- 10. El equipo puede demostrar todo el flujo de inicio a fin sin editar código durante la presentación.

Definición sencilla de “terminado”

Una persona nueva puede registrarse, iniciar sesión, ingresar datos, obtener un análisis, cerrar y volver a entrar, seguir viendo sus datos guardados y comparar el resultado actual con un resultado anterior.

2. Estado real que encontramos en GitHub

Área	Qué existe hoy	Qué falta	Meta del sábado
Frontend	En frontend_fernando ya existen login, registro, dashboard, gastos e index.	En feature/integrar-frontend solo aparecen index y gastos; faltan trasladar los avances nuevos y conectarlos a datos reales del usuario.	Login/registro + dashboard personal + historial funcionando desde la rama de integración.
Backend	Ya hay autenticación, perfil de usuario, transacciones, health, análisis y configuración de seguridad.	DashboardController está vacío; falta guardar historial de análisis y completar la unión con Data Science.	API única que atienda al frontend, guarde datos y devuelva resumen actual + histórico.
Base de datos	Ya existen usuarios y transacciones asociadas a cada usuario; hay migraciones V1, V2 y V3.	No existe todavía una tabla confirmada para guardar cada análisis financiero y poder compararlo en el tiempo.	MySQL persistente con usuarios, transacciones e historial de análisis.
Data Science	En frontend_fernando existe el módulo con modelos, predict.py y modo full_analysis.	No está consolidado en la rama de integración y debe confirmarse el contrato final y las pruebas.	Backend llama al análisis completo y recibe una respuesta estable.
OCI	El diseño se ha definido y el backend tiene health para comprobar disponibilidad.	Faltan base real, contenedores, secretos, URLs públicas y prueba completa en nube.	Frontend + Backend/DS + MySQL disponibles y probados desde Internet.

Hallazgo importante: el frontend avanzado está en otra rama

La rama frontend_fernando tiene archivos que no están todavía en feature/integrar-frontend. Esto explica por qué al revisar la rama de integración parecía que el login y el registro no existían, aunque Fernando sí los había trabajado.

- frontend_fernando: incluye 404.astro, dashboard.astro, gastos.astro, index.astro y login.astro.

- feature/integrar-frontend: actualmente muestra únicamente gastos.astro e index.astro en la carpeta de páginas.
- La solución no es seguir desarrollando en la rama antigua: hay que trasladar lo útil a una rama corta creada desde feature/integrar-frontend y hacer Pull Request.

Decisión recomendada

feature/integrar-frontend será el punto común de trabajo hasta que el MVP completo quede probado. main se actualiza al final, no durante el trabajo diario.

3. Qué está incompatible y cómo lo resolvemos

La principal dificultad no es que un área esté “mal”. La dificultad es que las partes se desarrollaron en momentos y ramas distintas y hoy esperan datos diferentes. Para terminar rápido, necesitamos una sola forma de comunicarnos.

3.1 El Backend será la fuente oficial de lo que recibe y devuelve la aplicación

Frontend solo habla con Backend. Data Science también se conecta a través de Backend. Esto evita que una modificación interna de Python obligue a cambiar la interfaz.

```
Frontend -> Backend -> Data Science
Frontend <- Backend <- Data Science
```

Regla del contrato

El equipo acuerda qué información necesita el producto. Backend publica ese formato en Swagger. Frontend usa exactamente ese formato. Si Data Science usa nombres distintos internamente, Backend hace la traducción.

3.2 Registro: hay un ajuste concreto que hacer

El formulario de Fernando incluye Nombre y Apellido. El registro actual del Backend recibe Nombre, correo, contraseña, país y moneda; no recibe Apellido.

- Para llegar al sábado sin ampliar alcance, la recomendación es quitar “Apellido” del envío del Frontend y conservar solo “Nombre”, salvo que el equipo decida hoy que Apellido es obligatorio.
- Si se decide conservar Apellido, Backend y la base de datos deben agregarlo de forma explícita y probar la migración. No dejar el campo “a medias”.

3.3 Análisis: el Backend actual y Data Science usan información distinta

El análisis Java que ya funciona usa ingreso, gasto, ahorro, deuda y moneda. El modo full_analysis de Data Science trabaja con ingreso, nivel de endeudamiento, frecuencia de ahorro y transacciones del período. Frontend también está más cerca de este segundo enfoque.

Recomendación para el MVP

Mantener un único endpoint de análisis, pero ampliar su solicitud para incluir la información necesaria del período y las transacciones. Backend transforma esos datos al formato que espera Data Science. El análisis Java actual se conserva como respaldo mientras la integración se estabiliza.

3.4 Moneda: decisión que no debe quedar implícita

El usuario selecciona moneda al registrarse. Data Science documenta el análisis completo con montos en moneda local, pero parte de sus datos de entrenamiento está expresada en USD. El equipo de Data Science debe confirmar el martes cómo tratará la moneda. No se debe inventar una conversión en Backend sin una regla definida.

4. Cómo debe funcionar la cuenta de cada usuario

El login ya guarda un token de sesión en el navegador. El siguiente paso es usar esa sesión para que todas las consultas privadas pertenezcan a la persona que inició sesión.

1. Registro: se crea el usuario con país y moneda.
2. Login: Backend valida las credenciales y entrega la sesión.
3. Perfil: el Frontend consulta el perfil real del usuario, no datos escritos a mano.
4. Transacciones: cada movimiento nuevo se guarda asociado al usuario autenticado.
5. Consulta: el usuario ve solamente sus transacciones.
6. Análisis: el cálculo se realiza para ese usuario y ese período.
7. Historial: cada resultado importante se guarda para poder compararlo después.
8. Logout: el Frontend elimina la sesión local y ya no permite consultar información privada.

Qué ya ayuda a lograr esto en Backend

- GET /api/v1/users/profile ya trabaja con el usuario autenticado.
- POST /api/v1/transactions crea una transacción para el usuario autenticado.
- GET /api/v1/transactions devuelve las transacciones de ese usuario.
- La entidad Transaction ya guarda fecha y relación con usuario.

5. Base de datos final del MVP

La base final debe ser MySQL. H2 seguirá siendo útil solo para pruebas locales. El producto desplegado debe guardar datos de forma permanente.

Información	Estado hoy	Qué debe quedar el sábado
Usuarios	Existe tabla/entidad de usuarios.	Registro, login, país, moneda y contraseña segura.
Transacciones	Existe y cada transacción tiene usuario y fecha.	Ingresos/gastos consultables por usuario y período.
País y moneda	Ya forman parte del usuario.	Se usan en perfil y presentación de montos.
Historial de análisis	No está confirmado en GitHub.	NUEVO: guardar una fotografía del resultado de cada período para comparar.

Tabla nueva recomendada: historial de análisis

No hace falta guardar todo el JSON si eso complica el tiempo. Para el MVP basta guardar los datos que permitan reconstruir la comparación.

- Usuario al que pertenece el análisis.
- Fecha o período (por ejemplo, agosto 2026).
- Ingreso mensual.
- Nivel de endeudamiento.
- Frecuencia de ahorro.
- Total de gastos.
- Ahorro estimado.
- Score o perfil financiero final.
- Principales categorías o un resumen del resultado, si el equipo lo necesita.

Comparación mínima que debe mostrar el dashboard

Período actual vs período anterior: total de gastos, ahorro estimado, nivel de endeudamiento, perfil/score y categorías principales. Si solo existe un período, mostrar “Aún no hay historial suficiente”.

6. Qué debe hacer cada frente

6.1 Frontend - Fernando

- Dejar de agregar cambios nuevos en frontend_fernando.
- Crear una rama corta desde feature/integrar-frontend y trasladar login, registro, dashboard y cualquier componente nuevo que sí se vaya a usar.
- Alinear el registro con Backend: resolver el campo Apellido.
- Usar la sesión del usuario en todas las llamadas privadas.
- Conectar perfil, transacciones, análisis e historial a datos reales; eliminar datos de demostración donde ya exista API.
- Agregar cierre de sesión.
- Mostrar comparación actual vs histórico en el dashboard.
- Reemplazar localhost fijo por una dirección configurable para OCI.
- Probar errores: credenciales incorrectas, sesión vencida, Backend caído, usuario sin historial.
- Entregar pnpm test y pnpm build sin errores.

Terminado para Frontend

Un usuario puede registrarse, iniciar sesión, ver su nombre/datos, registrar o consultar movimientos, ejecutar análisis, ver comparación histórica y cerrar sesión.

6.2 Backend - Integración y datos (tu frente)

- Definir en Swagger el formato oficial de registro, transacciones, análisis y dashboard.
- Completar la integración del análisis con Data Science sin exponer detalles internos de Python al Frontend.
- Mantener el análisis Java actual como respaldo temporal si el motor de Data Science falla.
- Guardar las transacciones y el historial de análisis en MySQL.
- Completar DashboardController para devolver resumen actual e historial del usuario.
- Asegurar que análisis y dashboard trabajen con el usuario autenticado, no con un usuario enviado manualmente.
- Agregar la nueva migración de base de datos para historial y probarla desde cero.
- Preparar el backend y Data Science para ejecutarse en OCI.
- Mantener /api/v1/health disponible para comprobar el servicio.

Terminado para Backend

Swagger muestra los endpoints finales; la información se guarda y se recupera por usuario; análisis e historial responden con datos reales y el sistema vuelve a funcionar después de reiniciar contra MySQL.

6.3 Backend - Seguridad (compañero)

- Revisar registro y login de punta a punta con el Frontend de Fernando.
- Validar creación y uso del token de sesión.
- Definir qué rutas son públicas: health, login, registro y documentación; proteger perfil, transacciones, análisis e historial.
- Comprobar que Usuario A nunca pueda consultar datos de Usuario B.

- Probar respuestas de acceso no autorizado y sesión/token inválido.
- Revisar CORS en local y luego actualizarlo con la URL real del Frontend en OCI.
- Verificar que contraseñas y JWT_SECRET no queden escritos en GitHub.
- Revisar manejo de cierre de sesión del lado Frontend para el MVP.
- Documentar en una página qué queda protegido y cómo probarlo.

Cómo te ayuda directamente

Mientras tú trabajas en análisis, historial, MySQL e integración con Data Science, él se hace responsable de que login, permisos, CORS y secretos queden correctos. Así no tienes que detener la integración para revisar seguridad cada vez.

6.4 Data Science

- Mover o integrar el módulo final desde frontend_fernando hacia una rama creada desde feature/integrar-frontend.
- Confirmar que predict.py y los modelos .pkl funcionan fuera del notebook.
- Confirmar el modo full_analysis como entrada principal del Backend o proponer el ajuste final el martes.
- Confirmar campos obligatorios y respuesta estable.
- Confirmar cómo se tratarán moneda y montos.
- Probar al menos: perfil saludable, perfil en riesgo, transacción desconocida, lista vacía y datos límite.
- Entregar requirements.txt reproducible y una prueba simple que cualquiera del equipo pueda ejecutar.
- No cambiar el contrato después del miércoles salvo error crítico.

Terminado para Data Science

Backend puede enviar un análisis completo y recibir siempre una respuesta con perfil, gastos/categorías y recomendaciones, o un error controlado.

6.5 OCI - tu frente después de cerrar local

- Usar MySQL persistente en OCI; H2 no es la base final.
- Empaquetar Backend + Data Science para que ejecuten de la misma forma que en local.
- Subir la imagen al registro de contenedores de OCI y desplegarla en Container Instances.
- Configurar DB_URL, DB_USERNAME, DB_PASSWORD y JWT_SECRET fuera del código.
- Publicar Frontend. Si el build estático está estable, usar Object Storage + API Gateway; si el Dockerfile del Frontend resulta más rápido de validar, usarlo como plan alternativo.
- Cambiar la dirección del Backend en Frontend por la URL pública.
- Actualizar CORS con la URL pública del Frontend.
- Probar health, login, transacciones, análisis e historial desde Internet.

Terminado para OCI

La demo funciona desde una URL pública y mantiene los datos al reiniciar la aplicación.

6.6 Documentación y QA - responsabilidad compartida

- Actualizar README: dejar de decir que el proyecto está en “fases iniciales”.
- Agregar un diagrama sencillo del flujo Usuario -> Frontend -> Backend -> Data Science -> MySQL.
- Documentar cómo ejecutar Frontend, Backend y Data Science.
- Documentar variables necesarias sin incluir secretos reales.

- Guardar ejemplos de pruebas en Swagger/Postman.
- Preparar datos de demo y un guion de 5 a 7 minutos.
- Registrar errores encontrados y quién los corrige; el sábado no agregar funciones nuevas salvo necesidad crítica.

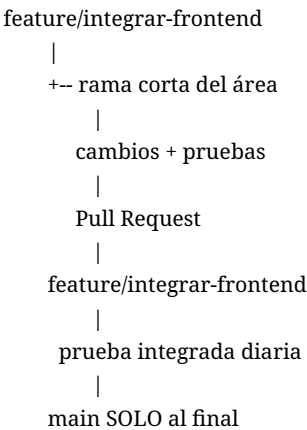
7. Cómo trabajar en GitHub sin volver a separar el proyecto

Rama común

feature/integrar-frontend es la rama de integración de esta semana. Nadie debe usar frontend_fernando como rama de desarrollo continuo y nadie debe trabajar directamente en main.

Frente	Rama corta sugerida
Frontend	feature/frontend-final
Backend seguridad	feature/backend-security
Backend análisis + historial	feature/backend-analysis-history
Data Science	feature/data-science-integration
OCI / Docker	feature/oci-deploy

Flujo diario recomendado:



- Antes de empezar: fetch + actualizar feature/integrar-frontend.
- Una tarea = una rama corta. Evitar una rama que dure toda la semana.
- Antes del Pull Request: pruebas del área.
- Después del merge: prueba rápida del flujo integrado.
- Si dos personas van a modificar el mismo archivo, acordarlo antes de empezar.

8. Plan de lunes a sábado

Día	Resultado obligatorio al cerrar el día	Quién debe empujar ese resultado
Lunes 17	Congelar alcance, compartir este plan, definir contrato único y ramas. No añadir nuevas ideas después de esta reunión.	Todos
Martes 18	Traer login/registro/dashboard a integración; resolver Apellido; cerrar contrato de análisis; DS integrado en rama; diseño/migración de historial definida.	Frontend + Backend + DS + Seguridad
Miércoles 19	Login real + perfil + transacciones por usuario + análisis Backend-DS funcionando local. Guardar el primer historial de análisis.	Frontend + Backend + DS + Seguridad

Jueves 20	Dashboard con datos reales y comparación actual vs período anterior. Prueba completa con dos usuarios y aislamiento correcto.	Frontend + Backend + Seguridad + QA
Viernes 21	MySQL y aplicación desplegados en OCI; Frontend apunta a URL pública; CORS y secretos listos; primera prueba E2E en nube.	OCI + Backend + Frontend + Seguridad
Sábado 22	Solo pruebas, correcciones, documentación, captura de evidencias y ensayo. Congelar funciones por la tarde y cerrar entrega por la noche.	Todos

Regla de calendario

Si el jueves por la noche el flujo local completo todavía no funciona, el viernes no se debe dedicar a mejoras visuales. La prioridad pasa a ser integrar y desplegar el flujo mínimo completo.

9. Pruebas obligatorias antes del sábado por la noche

Prueba	Resultado esperado	Responsable principal
Registro nuevo	Cuenta creada y usuario guardado en MySQL.	Frontend + Seguridad
Correo duplicado	Mensaje claro; no crea segundo usuario.	Backend + Seguridad
Login correcto/incorrecto	Correcto entra; incorrecto muestra error y no crea sesión.	Frontend + Seguridad
Aislamiento de usuarios	Usuario A no ve perfil, movimientos ni historial de B.	Seguridad
Crear transacción	Movimiento se guarda con fecha y usuario.	Backend
Consultar transacciones	Después de recargar/reiniciar, los movimientos siguen visibles.	Backend + Frontend
Análisis actual	Devuelve perfil/score, categorías y recomendaciones sin error.	Backend + DS
Guardar análisis	El resultado actual queda registrado en historial.	Backend
Comparación histórica	Con dos períodos, dashboard muestra diferencias de forma comprensible.	Backend + Frontend
Usuario nuevo sin historial	Dashboard no falla; explica que aún no hay comparación.	Frontend
Data Science falla	Backend devuelve error controlado o usa el fallback acordado.	Backend + DS
Backend test	mvnw test / compile termina correctamente.	Backend
Frontend build	pnpm test y pnpm build terminan correctamente.	Frontend
Data Science tests	Prueba de predict.py/modelos termina correctamente.	DS
Persistencia MySQL	Reinicio de la aplicación no borra usuarios, transacciones ni historial.	Backend + OCI
OCI público	Desde otra conexión se puede abrir Frontend y completar el flujo.	OCI + QA

10. Qué debe verse en Swagger al final

Swagger será la referencia común del equipo. Las rutas que ya existen deben mantenerse documentadas; las de dashboard/history se agregan cuando Backend las implemente.

Ruta	Uso	Estado/acción
POST /api/v1/auth/register	Crear cuenta.	Existe; alinear con Frontend.

POST /api/v1/auth/login	Iniciar sesión.	Existe; probar con Frontend.
GET /api/v1/users/profile	Ver perfil del usuario.	Existe; conectar al dashboard.
PUT /api/v1/users/profile	Actualizar perfil.	Existe; probar permisos.
POST /api/v1/transactions	Guardar movimiento.	Existe; conectar al Frontend.
GET /api/v1/transactions	Consultar movimientos propios.	Existe; conectar al Frontend.
POST /api/v1/analisis-financiero	Analizar período actual.	Existe; ampliar/ajustar contrato y proteger.
GET /api/v1/dashboard/summary	Resumen actual.	PROPUESTO: completar DashboardController.
GET /api/v1/dashboard/history	Histórico/comparación.	PROPUESTO: completar DashboardController.
GET /api/v1/health	Comprobar que Backend está vivo.	Existe y ya fue validado localmente.

11. Checklist de “sí podemos entregar”

- ☐ Todos los cambios importantes están en feature/integrar-frontend.
- ☐ No dependemos de frontend_fernando para ejecutar la demo.
- ☐ Registro y login funcionan contra Backend real.
- ☐ El usuario solo ve sus propios datos.
- ☐ Las transacciones se guardan en MySQL.
- ☐ El análisis usa el flujo Backend -> Data Science acordado.
- ☐ Cada análisis/período necesario queda guardado para historial.
- ☐ Dashboard muestra período actual y comparación con el anterior.
- ☐ Los secretos no están en GitHub.
- ☐ Backend, Frontend y DS pasan sus pruebas principales.
- ☐ La aplicación funciona en OCI con URL pública.
- ☐ Se probó el flujo desde un navegador limpio/otra conexión.
- ☐ README y documentación reflejan el estado real.
- ☐ Hay un guion de demo y datos preparados.
- ☐ main recibe el MVP probado mediante Pull Request final.

12. Mensaje para alinear al equipo

Texto sugerido para la reunión

“Ya no necesitamos seguir creando módulos por separado. Tenemos avances reales en Frontend, Backend y Data Science, pero están repartidos entre ramas y algunos contratos no coinciden. Desde hoy feature/integrar-frontend será el punto común. Backend será la referencia de lo que recibe y devuelve la aplicación; Frontend consumirá esa API y Backend traducirá lo necesario hacia Data Science. El objetivo del sábado no es tener más pantallas: es que un usuario se registre, inicie sesión, guarde sus datos, reciba un análisis, vuelva a entrar y compare su situación actual con su historial, todo funcionando en OCI.”

13. Decisiones que deben quedar cerradas el martes

1. ¿Apellido se elimina del registro o se agrega formalmente a Backend y MySQL?
2. ¿Cuál es el request y response definitivo del análisis que aparecerá en Swagger?

3. ¿Cómo confirma Data Science el manejo de moneda?
4. ¿Qué campos exactos se guardarán en historial de análisis?
5. ¿Qué rutas quedan protegidas después del login?
6. ¿Quién aprueba y hace merge de cada Pull Request a feature/integrar-frontend?
7. ¿Qué alternativa de publicación del Frontend en OCI usarán para no bloquear el viernes?

14. Base de este plan

Este documento reemplaza y unifica las guías previas de reunión, plan semanal e incompatibilidades. Se actualizó con la revisión del repositorio realizada el 17 de agosto de 2026.

- Rama feature/integrar-frontend: Backend, DataScient y financeai-frontend; el Frontend integrado todavía muestra solo index y gastos en src/pages.
- Rama frontend_fernando: contiene los avances adicionales de Frontend (login, dashboard, etc.) y el módulo financeai-data-science con predict.py/modelos.
- Backend de integración: AuthController, TransactionController, UserController y DashboardController; este último está vacío actualmente.
- Base existente: usuarios y transacciones, con transacciones relacionadas al usuario y fecha; migraciones V1, V2 y V3.
- OCI: se mantiene una arquitectura de cierre simple usando servicios actuales de Oracle Cloud; Container Instances permite ejecutar contenedores sin administrar servidores, MySQL HeatWave ofrece MySQL gestionado y Object Storage + API Gateway puede alojar contenido estático.